

자연어처리개론 기말 프로젝트

- GPT2 완성하기

노성환, 이다희

1. 사용한 기법

1.1 RoLA

1.2 Instruction Tuning

1.3 Task-specific Head 추가 및 튜닝 방식

1.4 Distillation

2. Task

2.1 Sentiment Analysis

2.1.1 모델 구조 및 튜닝 방식

2.1.2 결과

2.2 paraphrase detection

2.2.1 서론 및 튜닝 방식

2.2.2 결과

2.2.3 실패한 실험

2.3 Sonnet Generation

2.3.1 모델 구조 및 튜닝 방식

2.3.2 실험_1

2.3.2 실험_2

1. 사용한 기법

1.1 RoLA

LoRA (Low-Rank Adaptation) 기본 원리

LoRA는 대규모 모델 전체를 fine-tuning 하는 경우 발생하는 과도한 학습 비용과 GPU 메모리 사용 문제를 해결하기 위해 제안된 기법이다. 기존의 모델 weight를 그대로 보존한 채, 여기에 low-rank 행렬 두 개(A, B)를 덧붙이는 방식으로 작동한다. 이 방식은 특히 Transformer의 self-attention 구조 내 Q, K, V projection layer에 효과적이다.

LoRA 수식

$$W_{\text{eff}} = W + (\alpha / r) * (B @ A)$$

W: 기존 가중치 (학습되지 않음), $A \in \mathbb{R}^{r \times d}$, $B \in \mathbb{R}^{d \times r}$, α : scaling factor, r: rank (보통 4~8 정도)

LoRALinear 클래스 구성 및 설명

LoRA_adapter.py 코드 설명

1. 구성 요소 설명

- in_features, out_features: 입력 및 출력 차원
- r: low-rank 차원. 작을수록 파라미터 수가 감소하며, 보통 4~8 사용
- alpha: scaling factor로, low-rank 출력을 스케일링하여 학습 안정성을 조절하는 계수
- weight: 기존 모델에서 가져온 가중치. requires_grad=False로 설정하여 학습되지 않음
- A, B: 학습 대상이 되는 파라미터로, LoRA 방식의 핵심

2. forward() 로직

1. $\text{delta_W} = B @ A$: low-rank 보정값 계산
2. $W_{\text{eff}} = W + (\alpha / r) * \text{delta_W}$: 실제 적용될 가중치
3. $F.\text{linear}(x, W_{\text{eff}})$: nn.Linear와 동일한 방식으로 입력 x에 대해 선형 변환 수행

다른 클래스에서 활용

- convert_to_lora 메서드를 통해 기존 모델 구조에 손쉽게 LoRA를 적용할 수 있어 재활용성이 높다.
- 각 레이어별로 LoRA 모듈을 생성한 뒤, 원래의 파라미터는 고정(freeze)하여 효율적인 파인튜닝이 가능하다.

SonnetGPT class 내에서 convert_to_lora 메서드: LoRA 구조 적용

```
def convert_to_lora(self):
    """
    GPT2의 Q/K/V 프로젝트 레이어를 LoRALinear로 교체.
    """
    d = 768 # hidden dim

    for i in range(0, 12): # 상위 레이어만 적용
        layer = self.gpt.gpt_layers[i]

        # 기존 가중치를 복사
        q_weight = layer.self_attention.query.weight.data.clone()
        k_weight = layer.self_attention.key.weight.data.clone()
        v_weight = layer.self_attention.value.weight.data.clone()

        # LoRA로 대체
        layer.self_attention.query = LoRA_adapter.LoRALinear(d, d, r=4, alpha=16, base_weight=q_weight)
        layer.self_attention.key = LoRA_adapter.LoRALinear(d, d, r=4, alpha=16, base_weight=k_weight)
        layer.self_attention.value = LoRA_adapter.LoRALinear(d, d, r=4, alpha=16, base_weight=v_weight)
```

이 메서드는 기존 GPT2 모델의 0번 레이어부터 마지막(11번)까지의 attention projection layer를 LoRALinear로 바꾸어 적용한다. 초기에는 추론에 중요한 상위 레이어에만 LoRA를 적용해 효율적으로 튜닝을 시작하고, 하위 레이어는 그대로 둔다. 이후 학습이 안정되면 하위 레이어까지 점진적으로 확장해 나가며, 전체 모델의 표현력을 단계적으로 개선할 예정이다.

- q_weight, k_weight, v_weight는 기존의 nn.Linear weight를 그대로 복사하여 고정(freeze)된 상태로 사용된다.
- 각 weight는 LoRA가 정의한 low-rank 보정값과 더해져 학습 대상이 되므로, 기존 구조를 유지하면서도 **효율적인 파라미터 업데이트**가 가능하다.

sonnet_generation.py의 train 함수 중 lora의 layer와 Transformer block을 freeze하는 부분

우선 argparse 옵션을 정의한다. --freeze_lora_layers, --unfreeze_blocks 인자로 각 레이어, 블록 인덱스 리스트를 받는다.

```
parser.add_argument("--freeze_lora_layers", type=int, nargs='*', default=None,
                    help="LoRA 레이어 중 freeze할 레이어 인덱스 리스트 (예: --freeze_lora_layers 0 1 2)")
parser.add_argument("--unfreeze_blocks", type=int, nargs='*', default=None,
                    help="Transformer 블록 중 unfreeze할 인덱스 리스트 (예: --unfreeze_blocks 10 11)")
```

그런 다음 파이프라인을 설정한다.

- 훈련 단계별(phase)로 freeze_lora_layers와 unfreeze_blocks 값을 지정한다.
- start_epoch, end_epoch 등과 함께 각 단계에 적용할 설정을 구성한다.

```
if __name__ == "__main__":
    args = get_args()
    seed_everything(args.seed) # 재현성을 위한 random seed
    pipeline = [
        {
            "version": 0,
            "start_epoch": 0,
            "end_epoch": 30,
            "freeze_lora_layers": [0,1,2,3,4,5,6,7,8,9],
            "unfreeze_blocks": [10,11],
            "dataset": "distilled"
        },
        {
            "version": 1,
            "start_epoch": 30,
            "end_epoch": 60,
            "freeze_lora_layers": [0,1,2,3,4,5],
            "unfreeze_blocks": [8,9,10,11],
            "dataset": "distilled"
        },
    ],
```

실제로 train 함수 내에서 freeze하는 부분이다.

- args.freeze_lora_layers에 있는 LoRA 모듈의 requires_grad=False → 해당 레이어 freeze
- args.unfreeze_blocks에 있는 블록 파라미터의 requires_grad=True → 해당 block unfreeze

```
# Freeze 선택된 LoRA layer
if hasattr(args, "freeze_lora_layers"):
    for i in args.freeze_lora_layers:
        attn = model.gpt.gpt_layers[i].self_attention
        for lora_module in [attn.query, attn.key, attn.value]:
            lora_module.A.requires_grad = False
            lora_module.B.requires_grad = False

# Transformer block unfreeze
if hasattr(args, "unfreeze_blocks"):
    for i in args.unfreeze_blocks:
        for param in model.gpt.gpt_layers[i].parameters():
            param.requires_grad = True
```

왜 freeze를 통해 상위 레이어부터 LoRA를 적용하는가?

딥러닝 기반 언어 모델은 레이어가 깊어질수록 추상적인 정보를 처리하게 된다. 하위 레이어는 형태소, 구문, 문법 등 저수준 언어 정보를 담당하고, 상위 레이어는 문맥, 의미, 문장 구조 같은 고수준 표현을 처리한다.

따라서 LoRA를 상위 레이어부터 선택적으로 적용하는 것은, 모델의 출력에 큰 영향을 주는 표현력 중심의 부분만 우선적으로 조정함으로써 효율적인 성능 개선을 도모하는 전략이다. I 인자를 통해 몇 개의 상위 레이어에만 LoRA를 적용할지를 조절할 수 있으며, 이는 계산 자원을 절약하면서도 적은 수의 파라미터로 높은 효과를 얻기 위함이다.

추후에는 하위 레이어까지 점진적으로 확장해 나갈 예정이며, 초기에는 상위 레이어부터 단계적으로 학습을 진행한다.

1.2 Instruction Tuning

1. 개요 및 원리

Instruction Tuning은 언어 모델이 특정 작업을 명시적으로 이해하고 수행할 수 있도록, 지시문(instruction) 형태의 프롬프트를 입력으로 제공하며 학습시키는 기법이다. 이는 단순한 문장 생성을 넘어, 주어진 명령에 정확히 부합하는 출력을 생성하도록 모델을 학습시키는 방식의 fine-tuning이다.

예를 들어, GPT-2가 <task=...> <type=...>와 같은 구조화된 프롬프트를 이해할 수 있다고 가정하면, 모델은 해당 입력을 통해 어떤 작업을 수행해야 하는지 파악하며, 그에 정확히 부합하는 응답을 생성하게 된다.

Instruction Tuning에서는 이러한 (지시문+입력)-출력 쌍을 다수 제공함으로써, 모델이 지시문에 조건적으로 반응하는 방식으로 학습되도록 유도한다. 즉, 지시문에 따른 정답을 예측하도록 모델을 반복적으로 훈련함으로써, 지시문 기반의 통제 가능한 텍스트 생성을 가능하게 한다.

2. 프롬프트의 효과: 단일 태스크에서도 의미 있는 이유

비록 본 모델이 Sonnet Generation이라는 단일 작업에만 사용되더라도, instruction 형태의 프롬프트를 도입하는 것은 다음과 같은 측면에서 성능 향상에 기여할 수 있다:

- 지시문은 의미 신호(semantic signal)**로 작용하여, 출력의 스타일, 구조, 운율 등을 명확히 유도한다.
- Transformer 구조 특성상 입력 초반의 프롬프트는 전체 attention 분포에 영향을 주어, 더 일관성 있고 고품질의 생성이 가능해진다.
- <type=shakespearean>과 같은 지시는 학습 과정에서 **암묵적 라벨(implicit label)**로 작용하여, 학습 안정성을 높인다.

따라서 단일 태스크만 수행하는 경우에도, 프롬프트 기반 학습은 GPT 구조의 장점을 극대화하는 효과적인 전략이 될 수 있다.

Prompt 설계 (sonnet generation에서 예시)

```
prompt_tokens = [
```

```
"<task=Sonnet_Generation>", "<type=shakespearean>",
```

"<meter=iambic_pentameter>", "<rhyme=abab CDCDEFEFGG>"

]

이 구조화된 프롬프트는 다음과 같은 의미를 가진다:

- task=Sonnet_Generation: 모델이 시 생성 작업을 수행해야 함을 명시
- type=shakespearean: 셰익스피어식 소네트 형식을 따를 것
- meter=iambic_pentameter: 운율은 약강 5보격
- rhyme=abab CDCDEFEFGG: 스펜서식의 라임 스킴을 따름

이러한 instruction 이후에 이어질 소네트를 예측하도록 학습하는 것이 Instruction Tuning의 핵심 목표이다.

유의 사항

- 프롬프트가 정확하고 일관되며 구조화되어야 한다.

이를 통해 모델은 "무엇을 생성할지"를 명확히 이해하고 따를 수 있다.

코드 설명

SonnetGPT class 내의 forward 메서드: 프롬프트 삽입 및 입력 처리

Prompt 붙이기

1. Prompt 준비

```
prompt_tokens = [
    "<task=Sonnet_Generation>",
    "<type=shakespearean>",
    "<meter=iambic_pentameter>",
    "<rhyme=abab CDCDEFEFGG>"
]
prompt_text = " ".join(prompt_tokens)
prompt_ids = self.tokenizer.encode(prompt_text, add_special_tokens=False, return_tensors="pt").to(input_ids.device)
```

- 소네트 생성 태스크에 필요한 명시적 정보를 메타 프롬프트로 정의하고, 이를 토크나이즈한다.

- add_special_tokens=False로 지정하여 불필요한 특수 토큰 삽입을 방지한다.

2. 입력과 attention mask에 프롬프트 붙이기

```
batch_size = input_ids.size(0)
prompt_len = prompt_ids.size(1)
self.prompt_len = prompt_len # prompt_len 저장 (뒤에서 자르기 위함)
prompt_ids = prompt_ids.expand(batch_size, -1)
```

- 프롬프트는 모든 입력 시퀀스에 동일하게 삽입되므로, 배치 크기에 맞게 복제한다.

3. 입력 시퀀스 및 어텐션 마스크에 프롬프트 붙이기

```
input_ids = torch.cat([prompt_ids, input_ids], dim=1)
prompt_mask = torch.ones((batch_size, prompt_len), dtype=attention_mask.dtype, device=attention_mask.device)
attention_mask = torch.cat([prompt_mask, attention_mask], dim=1)
```

- 프롬프트 토큰을 입력 시퀀스 앞에 붙이고, 어텐션 마스크도 동일하게 확장하여 모델이 프롬프트를 인식하도록 한다.

4. GPT-2 백본을 통해 히든 스테이트 생성

GPT-2 backbone → hidden states (B, T, D)

```
hidden_states = self.gpt(
    input_ids=input_ids,
    attention_mask=attention_mask,
)
```

- 최종적으로 확장된 입력 시퀀스를 GPT-2에 전달하여 hidden state를 생성한다.

1.3 task-specific Head

Task-specific-Head 기본 원리

사전 학습된 GPT-2는 언어 생성(generative)을 목적으로 설계된 모델로, 주어진 문맥에 따라 다음 단어를 예측하는 데 최적화되어 있다. 이처럼 GPT-2는 각 시점마다 어휘 전체에 대한 확률 분포를 출력하는 Autoregressive 언어 모델이기 때문에, 입력 전체를 보고 하나의 판단을 내려야 하는 분류(classification) 작업에는 직접적으로 사용하기 어렵다.

이러한 한계를 해결하기 위해, GPT-2의 표현 능력은 그대로 유지하면서도 task에 따라 적절한 출력을 생성할 수 있도록 Task-specific Head를 추가하여 사용하였다. 이 구조는 GPT-2를 인코더

로 사용하고, 그 위에 과제별 출력 목적에 맞는 분류기나 생성기를 부착하는 방식이다.

구성

Task-specific Head는 GPT-2가 출력한 마지막 토큰의 hidden state를 입력으로 받아, 최종 출력값을 생성한다. 이 head를 간단한 2층 MLP 구조로 설계하였다.

```
[입력 문장] → GPT-2 → [마지막 토큰 벡터]
                → Linear → ReLU → Linear → [감정 or 유사 여부]
```

Sentiment Analysis: 5개 감정 클래스 중 하나 선택 (Linear-ReLU-Linear → 5-d output)

Paraphrase Detection: yes/no 선택 (Linear-ReLU-Linear → 2-d output)

작동 방식

1. 입력 문장 토큰나이징 및 인코딩

입력 문장을 GPT-2 tokenizer를 통해 토큰화하고, attention mask와 함께 GPT-2에 전달한다. 필요에 따라 task-specific 프롬프트를 앞에 붙이기도 한다.

```
outputs = gpt(input_ids=input_ids, attention_mask=attention_mask)
```

2. 문장 벡터 추출 (마지막 토큰 벡터)

GPT-2는 각 토큰에 대한 contextualized representation을 출력한다. 이 중 마지막 토큰의 hidden state를 문장 전체 의미를 대표하는 벡터로 사용한다.

```
last_token_hidden = outputs["last_token"] # shape: [batch_size, hidden_size]
```

3. Task-specific Head에 전달

이 벡터를 MLP로 구성된 Task-specific Head에 입력한다. Head는 Linear → ReLU → Linear 구조로 이루어져 있고, 이 과정을 통해 최종 logits을 출력한다.

```
x = classifier_1(last_token_hidden)
x = relu(x)
logits = classifier_2(x) # shape: [batch_size, num_labels]
```

4. 예측 및 손실 계산

출력된 logits은 클래스별 확률 값으로 변환되며, 정답 레이블과 비교하여 손실을 계산한다. 이 손실을 바탕으로 모델을 업데이트하며 학습이 이루어진다.

```
loss = F.cross_entropy(logits, labels)
```

※ 실제 구현에서는 위와 같은 classifier_1 → ReLU → classifier_2 방식 외에도, 동일 구조를 nn.Sequential로 구성한 self.mlp 형태로 사용하는 경우도 있다. 두 방식은 구조적으로 동일하며 task-specific head 역할을 수행한다.

Task-specific Head의 장점

1. 분류 목적에 맞춘 출력 가능성

GPT-2는 원래 단어 생성을 위한 확률 분포를 출력하지만, Task-specific Head를 추가함으로써 감정 분류, 문장 유사도 판별 등 문장 단위 판단에 적합한 출력 형식(logits)을 얻을 수 있다.

2. 구조 단순성 및 확장성

Head 구조는 단순한 MLP로 구성되어 있어 구현이 쉽고, task마다 출력 차원만 바꾸면 다양한 과제에 적용 가능하다.

예: 감정 분류(5-class), paraphrase 판별(2-class)

3. 사전학습 모델 재사용 가능

GPT-2의 언어 이해 능력을 그대로 유지하면서, 출력부만 추가 학습하면 되기 때문에 학습 효율이 높고, 전이학습에 유리하다.

4. 연산량 절감

특히 fine-tuning 범위를 head로 제한할 경우, 훈련 시간이 짧고, 연산 자원 소모가 적다. 이는 low-resource 환경에서 유리하다.

1.4 Distillation

Distillation 기본 원리

Knowledge Distillation은 원래 높은 성능을 가진 대형 모델(Teacher)의 지식을 소형 모델(Student)로 전달하여, 파라미터 수는 줄이면서도 성능을 유지하거나 향상시키는 학습 기법이다. 그 핵심 원리는 Teacher 모델이 제공하는 출력 정보를 Student가 모방함으로써 일반화 성능을 개선하는 데 있다.

일반적인 분류 태스크에서는 각 클래스에 대한 softmax 확률 분포(soft label)를 Student가 학습

하게 되며, 이를 통해 정답 이외의 클래스에 대한 Teacher의 "추론 경향성"까지 반영할 수 있다. 반면 언어 모델의 경우, soft label 접근은 제한적이므로 주로 hard label(출력 텍스트)을 직접 모방하는 방식으로 distillation이 수행된다.

Distillation 방식

- Teacher 모델: gpt-4.1
- Student 모델: SonnetGPT
- 학습 데이터 구성: 학습 데이터는 gpt-4.1 API로부터 생성된 소네트를 텍스트 파일로 저장한 형태이며, 기존 GPT2/data/sonnets.txt 형식을 따랐다. 각 샘플은 아래와 같은 구조로 구성되어 있다:

{index}

{... Sonnet ...}

{index}

...

학습 절차:

1. 사전 정의된 프롬프트에 대해 GPT-4로 데이터셋 생성
2. 생성된 데이터셋을 정답(label)으로 사용하여 SonnetGPT 학습
3. Cross-Entropy Loss 기반의 일반적인 Causal Language Modeling 방식으로 최적화

왜 soft label이 아닌 hard label을 사용했는가

일반적인 언어 모델 학습은 hard label—즉, 각 시점에서의 정답 토큰—을 기반으로 Cross-Entropy Loss를 계산하는 방식으로 이루어진다. 이는 학습 효율성과 구현 간편성, 그리고 soft label을 활용하기 위한 정보 접근의 제한(예: API 사용 시) 때문이기도 하다.

그러나 이론적으로는 soft label, 즉 각 토큰 위치에서의 전체 확률 분포를 사용할 수 있다면 더

욱 풍부한 학습 신호를 제공할 수 있다. 예를 들어, 다음 토큰으로 무엇이 올 수 있는지에 대한 확률적 힌트를 모델이 학습할 수 있으므로, 더 부드럽고 일반화 가능한 분포를 근사하게 된다.

예시

Hard label: 정답 토큰 "love"만을 정답으로 설정

Soft label: "love": 0.75, "like": 0.15, "adore": 0.05, ...

하지만 GPT-4 API는 이러한 soft label(확률 분포)을 제공하지 않기 때문에, 현실적으로는 hard label만을 사용하여 distillation을 수행하게 된다. 이와 같은 제약 하에서도 GPT-4의 출력은 이미 고품질 문장으로 구성되어 있기 때문에, hard label만으로도 효과적인 학습이 가능하다.

결론적으로, soft label은 이론적으로는 더 정밀한 학습을 가능하게 하지만, hard label은 현실적인 제약 아래에서도 충분히 실용적인 distillation 대안으로 기능한다.

장점

1. 소규모 모델도 고성능 모델의 표현력을 일부 모방 가능
2. 직접적인 데이터 수집 없이도 양질의 학습 데이터를 대량 확보할 수 있음
3. Shakespeare 스타일을 강제할 수 있음

데이터셋 생성 파이프라인

1. 시드 확보

- 속성 조합을 기반으로 총 1296개의 고유 시드를 생성
- 시드를 만든 이유:

중복 방지: 단순 반복 프롬프트로 유사한 소네트가 생성될 가능성이 높기 때문

다양성 확보: 시드마다 주제, 어조, 표현 기법 등이 달라져 다양한 시도 가능

2. 2행 생성

- 각 시드에 대해 gpt-4.1에게 소네트의 첫 2행만 생성하도록 요청
- 다양한 시작점을 확보해 전체 시의 다양성을 높임

- 총 7개의 문학적 속성(theme, tone, language_style, ...)에 대해 가능한 조합을 모두 생성
- 속성값을 기반으로 <theme: betrayal>, <tone: bitter> 와 같은 시드 프롬프트가 생성되며, 총 1296개의 조합이 results 리스트에 저장된다.

2. gpt-4.1을 활용해 Sonnet 2행 생성

```
def generate_sonnet_first_2_lines(client, prompt):
    system_prompt = "You are a poetic assistant. You generate Shakespearean sonnets in iambic pentameter with the o
    full_prompt = "\n".join([
        "<task=Sonnet_Generation>",
        "<type=shakespearean>",
        "<meter=iambic_pentameter>",
        "<rhyme=abab cdcd efef gg>",
        *prompt,
        "Write only the **first two lines** of a Shakespearean sonnet based on the attributes above",
        "Do not explain. Do not include any headings. Just output the two lines of the poem.."
    ])

    try:
        response = client.chat.completions.create(
            model="gpt-4.1",
            messages=[
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": full_prompt}
            ],
            temperature=0.7,
            max_tokens=100,
        )
        return response.choices[0].message.content
    except Exception as e:
        print(f"Error: {e}")
        return None
```

- 각 시드에 대해 GPT-4에 "소네트의 첫 두 줄"만 생성하도록 요청하는 함수
- 사전 정의된 <task=Sonnet_Generation>, <type=shakespearean> 등 task 메타 정보와 시드 프롬프트를 조합하여 입력
- 해당 결과는 GPT2/data/gpt4_2line_sonnet.jsonl 으로 저장

3. 전체 Sonnet 생성

```
def generate_sonnet_full_lines(client, given_sonnet):
    first_two_lines = given_sonnet
    full_prompt = f"""<task=Sonnet_Generation>
    <style=shakespearean>
    <meter=iambic_pentameter>
    <rhyme_scheme=abab CDCDEFEFGG>
    {first_two_lines}

    Output only the poem."""

    try:
        response = client.chat.completions.create(
            model="gpt-4.1",
            messages=[
                {"role": "user", "content": full_prompt}
            ],
            temperature=0.7,
            max_tokens=300,
        )
        return response.choices[0].message.content
    except Exception as e:
        print(f"Error: {e}")
        return None
```

- 앞서 생성된 2행을 기반으로 전체 14행을 생성
- 2행은 그대로 프롬프트에 삽입되며, GPT-4는 나머지 12행을 이어서 작성
- 해당 결과는 GPT2/data/Full_gpt4_distillation_sonnet_outputs.txt에 저장

4. 결과 저장 및 메모리 관리

```
# flush 함수: 결과를 파일에 저장
def flush_to_file(data, path):
    with open(path, "a", encoding="utf-8") as f:
        f.write("\n\n".join(data) + "\n\n") # 실제 줄바꿈 유지

# # index → output 매핑
index_to_sonnet = {}

with open(OUTPUT_FILE, "r", encoding="utf-8") as f:
    for line in f:
        item = json.loads(line)
        index_to_sonnet[item["index"]] = item["output"]
```

- 약 50개 단위로 생성 결과를 메모리에 임시 저장 (buffer)
- 일정 개수 도달 시 .txt 파일(OUTPUT_FILE_2)에 저장 후 메모리 비움

5. 마지막 couplet(13, 14행) 앞에 띄어쓰기 2번 추가

```
def format_sonnet_with_couplet_indent(sonnet_text: str) -> str:
    """
    14줄짜리 소네트를 받아 마지막 couplet(13, 14행) 앞에 띄어쓰기 2번 추가
    """
    lines = sonnet_text.strip().split("\n")
    if len(lines) == 14:
        # 마지막 2줄에 들여쓰기 추가
        lines[12] = "  " + lines[12]
        lines[13] = "  " + lines[13]
        return "\n".join(lines)
    return sonnet_text # 줄 수가 다르면 그대로 반환
```

- 마지막 2행(couplet) 앞에 들여쓰기를 추가하여, 시각적으로 구조를 강조하기 위한 후처리 단계